Playwright-Notes-5

- **Page Object Model in Playwright (POM)-** design pattern in test automation that creates an object repository for storing all locators-the page object will contain the representation of the page and the services the page provides via methods

- **Disadvantages of POM:**
    1. There is no separation between the test method and the AUT's (Application Under Test) locators
    2. The locators would be spread in multiple tests and would be difficult to maintain

- **Advantages of POM:**
    1. There is a clean separation between the test code and page-specific code such as locators
    2. There is a single repository for the services or operations the page offers rather than having these services scattered throughout the tests
    3. Page Object Model is a design pattern that has become popular in test automation for enhancing test maintenance and reducing code duplication
    4. The benefit is that if the UI changes for the page the tests themselves don't need to change, only the code within the page object needs to change

In VSCode:

1. **Create a separate folder known as pages in the same directory as the tests page**
2. **Under the pages folder create a folder called LoginPage.ts to perform the login action on the saucedemo website: "https://www.saucedemo.com/"**
3. **Write the following code:**

```
import {Locator, Page} from "@playwright/test" //1. import the page and locator components

export class LoginPage{ //2. create a class with a constructor and define the locators and page as readonly variables (variables that cannot be changed) and export it

    readonly page : Page; //page variable to hold the page object that is passed as an argument to the constructor and used to initialize the locators
```

```
    readonly usernameTextBox : Locator; //username field
    readonly passwordTextBox : Locator; //password field
    readonly loginButton : Locator; //login button

     constructor(page : Page){ //3. initialize the variables in
the constructor and assign the locators to the respective
variables using the page object that is passed as an argument to
the constructor
        this.page = page;
        this.usernameTextBox = page.locator("id=user-name");
        this.passwordTextBox = page.locator("id=password");
        this.loginButton = page.locator("id=login-button");

    }

     async openApplication(){ //5. create an asynchronous method
to open the application by navigating to the URL
        await this.page.goto("https://www.saucedemo.com/");
    }

     async login(usernameVal : string , passwordVal : string){
//4. create an asynchronous method to perform the login action by
filling the username and password and clicking the login button
        await this.usernameTextBox.fill(usernameVal);
        await this.passwordTextBox.fill(passwordVal);
        await this.loginButton.click();
    }



}
```

4. **Under the pages folder create a file called HomePage.ts which verifies that we are on the Home page after logging in and adds the selected item to the cart. In this file write down the following code:**

```
import {Locator, Page} from "@playwright/test" //1. import the
page and locator components
```

```
export class HomePage{ //2. create a class with a constructor and define the locators and page as readonly variables (variables that cannot be changed) and export it


    readonly page : Page; //page variable to hold the page object that is passed as an argument to the constructor and used to initialize the locators
      readonly homePageHeading : Locator; //home page heading locator to provide assertion in the test file to verify that we have navigated to the home page after login
     readonly backPackAddToCartButton : Locator; //Add to cart button for the backpack item
    readonly backpackRemoveButton : Locator; //Remove button for the backpack item that will be visible after adding the item to the cart
    readonly cartIcon : Locator; //cart icon that will have the number of items added to the cart as its text and will be used to navigate to the cart page

    constructor(page : Page){ //3. initialize the variables in the constructor and assign the locators to the respective variables using the page object that is passed as an argument to the constructor
        this.page = page;
        this.homePageHeading = page.getByText('Swag Labs');
                                this.backPackAddToCartButton    = page.locator('[data-test="add-to-cart-sauce-labs-backpack"]')
                                    this.backpackRemoveButton    = page.locator('[data-test="remove-sauce-labs-backpack"]')
                                                        this.cartIcon      = page.locator("id=shopping_cart_container")
    }

      async backPackAddToCart(){   //5. create an asynchronous method to add the backpack item to the cart by clicking on the add to cart button
        await this.backPackAddToCartButton.click();
    }

      async gotoCart(){ //6. create an asynchronous method to navigate to the cart page by clicking on the cart icon after adding the item to the cart
```

```
        await this.cartIcon.click();
    }


}
```

5. **Under the pages folder create a file called CartPage.ts which verifies that the selected item is added to the cart and is ready for checkout. In this file write down the following code:**

```
import { Locator, Page } from "@playwright/test" //1. import the page and locator components from the playwright test module to define the page and locators for the cart page

export class CartPage { //2. create a class with a constructor and define the locators and page as readonly variables (variables that cannot be changed) and export it

    readonly page: Page; //page variable to hold the page object that is passed as an argument to the constructor and used to initialize the locators
        readonly backpackItemLink: Locator; //locator for the backpack item link in the cart page to provide assertion in the test file to verify that the correct item has been added to the cart by checking the text of the backpack item link


    constructor(page: Page) { //3. initialize the variables in the constructor and assign the locators to the respective variables using the page object that is passed as an argument to the constructor
        this.page = page;
           this.backpackItemLink = page.getByRole('link', { name: 'Sauce Labs Backpack' })
    }


}
```

6. **Under the tests folder create a test script file called CartVerification.ts which imports all the 3 POM files we created in the pages folder and accesses their methods and properties via their objects in order to**

perform the test action. We write all the assertions in this file. In this file write down the following code:

```
import { test, expect } from "@playwright/test" //1. import the test and expect components from the playwright test module
import { LoginPage } from "../pages/LoginPage" //2. import the login page class from the pages folder to create an object of it and use its methods in this test file
import { HomePage } from "../pages/HomePage" //6. import the home page class from the pages folder to create an object of it and use its methods in this test file
import { CartPage } from "../pages/CartPage"; //13. import the cart page class from the pages folder to create an object of it and use its methods in this test file

test("Verification of Cart", async ({ page }) => {
        const loginPageObj = new LoginPage(page); //3. create an object of the login page class and pass the page object to its constructor to initialize the variables in the login page class
            await loginPageObj.openApplication(); //4. call the open application method of the login page class to navigate to the application URL (accessing methods of the LoginPage class using the loginPageObj object)
        await loginPageObj.login("standard_user", "secret_sauce"); //5. call the login method of the login page class to perform the login action by providing the username and password as arguments (accessing methods of the LoginPage class using the loginPageObj object)
         const homePageObj = new HomePage(page); //7. create an object of the home page class and pass the page object to its constructor to initialize the variables in the home page class
            await expect(homePageObj.homePageHeading).toHaveText("Swag Labs"); //8. provide an assertion to verify that we have navigated to the home page after login by checking the text of the home page heading
                await homePageObj.backPackAddToCart(); //9. call the backPackAddToCart method of the home page class to add the backpack item to the cart by clicking on the add to cart button (accessing methods of the HomePage class using the homePageObj object)
            await expect(homePageObj.cartIcon).toHaveText("1"); //10. provide an assertion to verify that the item has been added to the cart by checking the text of the cart icon which should have the number of items added to the cart as its text
          await expect(homePageObj.backpackRemoveButton).toBeVisible(); //11. provide an assertion to verify that the remove button for the
```

```
backpack item is visible after adding the item to the cart by checking
the visibility of the remove button locator
        await homePageObj.gotoCart(); //12. call the gotoCart method of
the home page class to navigate
         const cartPageObj = new CartPage(page); //13.create an object
of the cart page class and pass the page object to its constructor to
initialize the variables in the cart page class
          await expect(cartPageObj.backpackItemLink).toHaveText("Sauce
Labs Backpack"); //14. provide an assertion to verify that the correct
item has been added to the cart by checking the text of the backpack
item link in the cart page which should be the name of the item added
to the cart

})
```

Note: whenever a function/method returns a promise we need to use the await keyword and whenever we use the await keyword, we need to declare that function as an asynchronous function using async keyword

Note: Assertions are always placed in the test script file in the tests folder and not the POM file in the pages folder as POM represents the components and services of a page via methods and should not contain code related to what is being tested/validated (assertions)

Page Object Model (POM) in Playwright

What Is It?

A **design pattern** — a smart way to *organize* your test code so it doesn't turn into a mess.

> Think of it as **creating a remote control for each page of your app**. Instead of hardcoding every button and input everywhere, you build one tidy controller per page and reuse it across all your tests.

The Problem It Solves

Without POM — messy and fragile:

```
// Test 1
await page.fill('#email', 'user@test.com');
await page.fill('#password', 'secret');
await page.click('#login-btn');

// Test 2
await page.fill('#email', 'admin@test.com');
await page.fill('#password', 'admin123');
await page.click('#login-btn');

// Test 3
await page.fill('#email', 'guest@test.com');
await page.fill('#password', 'guest');
await page.click('#login-btn');
```

Now imagine the login button's ID changes from `#login-btn` to `#submit-btn`.

```
You have to hunt down and fix every single test. 😩
```

With POM — clean and maintainable:

```
// Fix it in ONE place — all tests instantly work again ✅
```

The Core Idea

One class per page. That class owns everything about that page.

```
Your App                    Page Objects
─────────────────           ──────────────────────
Login Page          →       LoginPage class
Dashboard Page      →       DashboardPage class
Settings Page       →       SettingsPage class
Checkout Page       →       CheckoutPage class
```

Building a Page Object — Step by Step

Step 1 — Create the Page Object class

```
// loginPage.js
class LoginPage {

  constructor(page) {
    this.page = page;

    // All the elements on this page live here
    this.emailInput    = page.locator('#email');
    this.passwordInput = page.locator('#password');
    this.loginButton   = page.locator('#login-btn');
    this.errorMessage  = page.locator('.error-msg');
  }

  // All the actions on this page live here
  async goto() {
    await this.page.goto('/login');
  }

  async login(email, password) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  async getErrorMessage() {
    return this.errorMessage.textContent();
  }
}

module.exports = LoginPage;
```

Step 2 — Use it in your tests

```
// login.test.js
const LoginPage = require('./loginPage');

test('successful login', async ({ page }) => {
  const loginPage = new LoginPage(page);

  await loginPage.goto();
  await loginPage.login('user@test.com', 'secret');
```

```
  await expect(page).toHaveURL('/dashboard');
});

test('wrong password shows error', async ({ page }) => {
  const loginPage = new LoginPage(page);

  await loginPage.goto();
  await loginPage.login('user@test.com', 'wrongpassword');

  const error = await loginPage.getErrorMessage();
  expect(error).toBe('Invalid credentials');
});
```

See how clean the tests are? **No selectors, no messy details — just plain readable actions.**

A Real World Analogy

Imagine a **TV remote control**.

- The remote is your **Page Object**
- The buttons (Volume, Channel, Power) are your **locators**
- Pressing a button is calling an **action method**
- You don't need to know how the TV works internally — you just press buttons

If the TV's internal wiring changes, you update the remote once — not every person in the house.

Full Example — Multiple Pages Working Together

```
// pages/LoginPage.js
class LoginPage {
  constructor(page) {
    this.page = page;
    this.emailInput = page.locator('#email');
    this.passwordInput = page.locator('#password');
    this.loginButton = page.locator('#login-btn');
  }

  async goto() { await this.page.goto('/login'); }
```

```
  async login(email, password) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
}

// pages/DashboardPage.js
class DashboardPage {
  constructor(page) {
    this.page = page;
    this.welcomeMessage = page.locator('.welcome');
    this.logoutButton   = page.locator('#logout');
    this.profileLink    = page.locator('#profile-link');
  }

  async logout() {
    await this.logoutButton.click();
  }

  async getWelcomeText() {
    return this.welcomeMessage.textContent();
  }
}

// tests/userFlow.test.js
test('user logs in and sees welcome message', async ({ page }) => {
  const loginPage     = new LoginPage(page);
  const dashboardPage = new DashboardPage(page);

  await loginPage.goto();
  await loginPage.login('user@test.com', 'secret');

  const welcome = await dashboardPage.getWelcomeText();
  expect(welcome).toBe('Welcome back, User!');
});
```

The test reads like a **story** — no messy selectors, just clear actions.

Folder Structure

```
project/
│
├── pages/              ← All your Page Objects live here
│   ├── LoginPage.js
│   ├── DashboardPage.js
│   ├── CheckoutPage.js
│   └── SettingsPage.js
│
├── tests/              ← Your actual tests live here
│   ├── login.test.js
│   ├── checkout.test.js
│   └── settings.test.js
│
└── playwright.config.js
```

What Goes Inside a Page Object

```
Page Object Class
│
├── constructor()     ← Set up all locators (elements) for this page
│
├── Locators          ← The elements: buttons, inputs, links, text
│
├── Navigation        ← goto(), navigate(), open()
│
├── Actions           ← login(), fillForm(), clickSubmit(), search()
│
└── Getters           ← getTitle(), getErrorMessage(), getItemCount()
```

The Golden Rules of POM

Rule	Why
One class per page	Keeps things organized and focused
Locators only in the Page Object	Change selector once, fixes everywhere
No assertions inside Page Objects	Page Objects *do things*, tests *check things*

Methods should describe actions	`login()` not `fillEmailThenPasswordThenClick()`
Keep tests readable like English	`loginPage.login(email, pass)` tells a story

No Assertions in Page Objects ⚠️

```
// ❌ Wrong — assertion inside Page Object
async login(email, password) {
  await this.emailInput.fill(email);
  await this.loginButton.click();
  expect(page).toHaveURL('/dashboard'); // ← Don't do this here
}

// ✅ Right — assertion stays in the test
test('login works', async ({ page }) => {
  await loginPage.login('user@test.com', 'pass');
  expect(page).toHaveURL('/dashboard'); // ← Belongs here
});
```

Page Objects are **workers**. Tests are **inspectors**. Don't mix the two.

Side-by-Side Comparison

	Without POM	With POM
Selectors	Scattered everywhere	Defined once, in one place
When UI changes	Fix every test file	Fix one Page Object
Test readability	Messy, technical	Clean, readable like English
Reusability	Copy-paste everywhere	Import and reuse

Maintenance	Painful	Easy

The Simple Mental Model

Each page of your app gets its own "assistant".

That assistant knows everything about that page — where the buttons are, how to fill the forms, how to navigate. Your tests just *talk to the assistant* instead of wrestling with raw selectors.

Change the page? Update the assistant. Your tests never need to know.

- **Custom Fixtures in Playwright-** Playwright test is based on test fixtures-test fixtures are used to establish an environment for each test, giving the test everything it needs and nothing else-test fixtures are isolated between tests
- To create your own fixture use test.extend() to create a new test object that will include it
- Custom created fixtures can be used with POM and over before/after hooks
- There are 2 types of fixtures: Test fixture and Worker fixture
- The test fixture gets requested by test and reruns per test script
- The worker fixture runs only once per test file

In plain terms, **fixtures are just "setup that gets shared across tests."**

Think of it like a coffee shop that preps certain things before opening — the espresso machine is warmed up, cups are stacked, milk is frothed — so each barista (test) doesn't have to do all that from scratch.

Built-in Fixtures (Playwright gives you these free)

Fixture	What it is
`page`	A single browser tab
`browser`	The whole browser instance (Chrome, Firefox, etc.)
`context`	A browser "profile" — cookies, storage, etc. Isolated from other contexts
`request`	For making API/HTTP requests directly (no browser UI)

`browserName`	The name of the current browser (`'chromium'`, `'firefox'`, `'webkit'`)

In VSCode-

1. Create a folder called **fixtures** in the **Playwright-Practice directory**
2. Create a file called **MyCustomFixture.ts** in the **fixtures** folder
3. Import test component as baseTest (baseTest is your own custom text fixture) from playwright like so:

```
import {test as baseTest} from "@playwright/test";
```

4. Add the below line:

```
type MyFixtures = {
    fixture1:any;

}

type MyWorkerFixtures = {
    workerFixture1 : any;

}
```

Explanation of the above code snippet:

These are just TypeScript type definitions that describe the shape of your fixtures before you actually create them.

MyFixtures — describes your regular (per-test) fixtures:
type MyFixtures = {
fixture1: any; // a fixture that resets for every test (test-scoped fixture)
}

MyWorkerFixtures — describes your worker-scoped fixtures:

```
type MyWorkerFixtures = {
   workerFixture1: any;  // a fixture shared across all tests in a file (worker-scoped fixture)
}
```

any is just a placeholder — in a real project you'd replace it with actual types:

```
type MyFixtures = {
   loggedInPage: Page;      // a Playwright Page
   authToken: string;       // a string
     testUser: { id: number, name: string }  // an object
}
```

5. Add the below line:

```
export          const          test          =
baseTest.extend<MyFixtures                    ,
MyWorkerFixtures>({

    fixture1: async({}, use: (value: any) =>
Promise<void>) => {
        const fixture1 = "I am Fixture 1 ";
         console.log("Before part of Fixture
1");
        await use(fixture1);
          console.log("After part of Fixture
1");
    },

    workerFixture1 : [async({}, use: (value:
any) => Promise<void>)=>{
         const workerFixture1 = "I am worker
Fixture 1 ";
          console.log("Before part of Worker
Fixture 1");
        await use(workerFixture1);
           console.log("After part of Worker
Fixture 1");
    },{scope:"worker"}]
```

```
})
```

Explanation of the above code snippet:

The outer shell:

ts

```
export const test = baseTest.extend<MyFixtures, MyWorkerFixtures>({
  ...
})
```

- **Takes `baseTest` (Playwright's original) and adds your custom fixtures on top**
- **`<MyFixtures, MyWorkerFixtures>` tells TypeScript what shapes to expect**
- **`export` so your test files can import this custom `test`**

`fixture1` — a regular test-scoped fixture:

ts

```
fixture1: async ({}, use: (value: any) => Promise<void>) => {
  const fixture1 = "I am Fixture 1"

   console.log("Before part of Fixture 1")  // runs BEFORE the test
  await use(fixture1)                    // test runs HERE
   console.log("After part of Fixture 1")   // runs AFTER the test
}
```

- **`{}` — no other fixtures needed (empty destructure)**
- **`use` — the function that passes your value into the test. Everything before `use` is setup, everything after is teardown (cleanup after test is done)**

- Think of `await use(fixture1)` as the pause point — the test runs while it's paused here

---

`workerFixture1` — a worker-scoped fixture:

```
workerFixture1: [async ({}, use) => {
  const workerFixture1 = "I am worker Fixture 1"

  console.log("Before part of Worker Fixture 1")
  await use(workerFixture1)
  console.log("After part of Worker Fixture 1")

}, { scope: "worker" }]  // 👈 the only difference
```

- Wrapped in an array `[ async fn, { scope: "worker" } ]` — this is how you tell Playwright it's worker-scoped
- Created once and shared across all tests in the file, instead of resetting each test

---

Visualizing the execution order:

Say you have 2 tests in a file:

```
Worker Fixture setup    ← runs once
 Test 1:
   Fixture1 setup
   test runs
   Fixture1 teardown
 Test 2:
   Fixture1 setup
   test runs
   Fixture1 teardown
Worker Fixture teardown ← runs once
```

`use: (value: any) => Promise<void>` — what does this type annotation mean?

It's just TypeScript saying: *"`use` is a function that takes any value and returns a Promise"*. You don't normally need to write this explicitly — Playwright infers it — but it's fine to be explicit for clarity.

In short:

Part	What it does
`async ({}, use)`	setup function with no dependencies
`await use(value)`	hands the value to the test
code before `use`	setup / before-test logic
code after `use`	teardown / after-test logic
`[fn, { scope: 'worker' }]`	marks it as worker-scoped

Teardown simply means cleanup after the test is done.

Think of it like cooking:

Phase	Cooking	Fixture
Set	Get out pots, chop	Create a user, open a

u p	**veget ables**	**browser, log in**
T e s t r u n s	**Actual ly cook the meal**	**Your test does its thing**
T e a r d o w n	**Wash dishe s, put things away**	**Delete the user, close the browser, log out**

In a fixture, teardown is everything AFTER `await use()`:

```
fixture1: async ({}, use) => {


    // SETUP — runs before the test

    const db = await connectToDatabase()
```

```
    const user = await db.createUser({ name: "John" })


  // TEST RUNS HERE

  await use(user)


    // TEARDOWN — runs after the test

                    await db.deleteUser(user.id)
// 👈 cleanup

                    await db.disconnect()      // 👈 cleanup

}
```

Why is teardown important?

Without it, tests leave a mess behind:

- **Users pile up in the database**
- **Browser windows stay open**
- **Files don't get deleted**
- **One test's leftover data**

breaks the next test

Teardown ensures each test starts fresh with a clean slate.

In your specific code:

console.log("Before part of Fixture 1") // setup

await use(fixture1) // test runs

console.log("After part of Fixture 1") // teardown (cleanup would go here)

The `console.log` after `use` is just demonstrating *where* teardown lives — in a real app you'd put actual cleanup code there.

6. Now export this fixture component into another file called PracticeFixture.spec.ts (create this

in the test folder) and add the following code:

```
import {test} from "../fixtures/MyCustomFixture"


test("Practice Test 1" , async({fixture1, workerFixture1})=>{

console.log(fixture1);

console.log(workerFixture1);

})



test("Practice Test 2" , async({fixture1})=>{

console.log(fixture1);


})
```

Explanation in VSCode:This is a test file that uses the custom fixtures you built earlier. Let me break it down:

The import:

import { test } from "../fixtures/MyCustomFixture"

- **Importing YOUR custom `test` (not Playwright's default)**
- **This is what gives you access to `fixture1` and `workerFixture1`**
- **If you imported from `@playwright/test` instead, those fixtures wouldn't exist**

Test 1 — uses BOTH fixtures:

test("Practice Test 1", async ({ fixture1, workerFixture1 }) => {

```
    console.log(fixture1)
// prints "I am Fixture
1"


console.log(workerFixt
ure1) // prints "I am
worker Fixture 1"

})
```

- Asks for both fixtures by destructuring them `{ fixture1, workerFixture1 }`
- Playwright sees what you asked for and automatically sets them up before the test runs

---

Test 2 — uses ONLY `fixture1`:

```
test("Practice Test 2",
async ({ fixture1 }) => {

    console.log(fixture1)
// prints "I am Fixture
1"

})
```

- **Only asks for `fixture1`**
- **`workerFixture1` is not requested, but still alive in the background (worker scope lives for the whole file)**

What the console output looks like when you run this file:

Before part of Worker Fixture 1 ← workerFixture1 setup (once)

Before part of Fixture 1 ← fixture1 setup (Test 1)

I am Fixture 1 ← Test 1 runs

I am worker Fixture 1

After part of Fixture 1 ← fixture1 teardown (Test 1)

Before part of Fixture 1 ← fixture1 setup (Test 2)

I am Fixture 1
← Test 2 runs

After part of Fixture 1
← fixture1 teardown (Test 2)

After part of Worker Fixture 1 ← workerFixture1 teardown (once)

Key takeaway:

	Test 1	Test 2
`fixture1`	✅ used	✅ used
`workerFixture1`	✅ used	❌ not requested

Playwright is smart — it only sets up fixtures a test actually asks for. If Test 2 doesn't ask for `workerFixture1`, Playwright won't run its

setup/teardown for that test.

Output of Practice Test 1:

Before part of Fixture 1

Before part of Worker Fixture 1

I am Fixture 1

I am worker Fixture 1

After part of Fixture 1

After part of Worker Fixture 1

Output of Practice Test 2:

Before part of Fixture 1

I am Fixture 1

After part of Fixture 1

What are test and worker-scoped fixtures:

Fixtures by Scope

This controls how long the fixture lives:

Scope	Lives for...	Use when...
`test` (default)	One test only	Most things — page, login, etc.
`worker`	The whole test file run	Expensive setup like DB connection, auth token